

External Procedural Distillation: Natural-Language Teaching into Verified Procedural Memory with a Frozen Student

Matthew Anderson

github.com/sheriffoftilt/ver/semant-vm

September 2026 (preprint; not peer reviewed)

Abstract

We study whether a system whose only resident model is a small, frozen language model can acquire new procedural capability from a larger model without any change to its own neural weights. Three roles must be kept apart: the verified procedure library is the capability, a temporary 120B model is the acquirer, and the frozen 1.5B model is a fixed retrieval front end that maps requests onto stored procedures. In the Stateful Semantic Virtual Machine (SEMVM), neural models only propose and never hold state: the world lives in a SQLite store that models reach only through typed queries, and durable state, execution, and every learning decision are deterministic. A hosted teacher (gpt-oss-120b) teaches a skill in natural language. A transactional teaching controller has each lesson line grounded into a typed plan (by a hosted grounder), executes it inside a sandboxed SQLite world, and admits a demonstration as evidence only if it is complete and committed. A deterministic anti-unifier proposes a parameterized program, and an evidence verifier (replay, counterfactual argument substitution, state perturbation, negative cases, rollback injection, determinism) decides whether it may be activated in a persistent procedure library. The hosted models are then destroyed, and the skill is used from fresh processes under a network guard, with a frozen Qwen2.5-Coder-1.5B model interpreting requests behind deterministic checks. We report a seven-experiment preregistered lineage in a synthetic, bounded personal-organizer domain. In the final experiment, two resource changes—a teacher example-value pool of 4 instead of 2 and a teacher output budget of 8,192 instead of 4,096 tokens—raised the hosted-teacher/hosted-grounder arm to 70/72 exactly learned procedures pooled over two independent development draws (36/36 and 34/36), with zero wrong committed actions; the parent experiment had 31/36 on its own draw. The sealed LOCKED evaluation passed every behavioral acquisition, grounding, safety, and reuse criterion, but failed the preregistered proof-of-teacher-disconnection condition because teardown was probed before the hosted endpoint had fully disappeared. Behaviorally, LOCKED produced 23/23 exactly learned procedures and 20/20 exact scenarios, with zero network attempts logged by the post-handoff network guard; it is nonetheless not a clean pass, and we treat the permissive teardown check that caused this as a finding about verifying the verification code itself. The results support a narrow claim: SEMVM can accumulate externally stored, verified procedural capability acquired with assistance from a temporary teacher. The teacher is in the acquisition causal chain, not the execution causal chain, and continued growth of the library currently depends on access to a teacher or other source of usable demonstrations.

1 Introduction

A deployed model that must learn new, reusable behaviors faces a familiar trade-off. Updating its weights risks interference with what it already does [14, 19]; distilling from a larger model transfers behavior through training data or soft targets [11–13] but requires a training run, and the transferred behavior is not individually inspectable. A third option is to keep the model frozen and put the learned capability somewhere else.

This paper describes one concrete version of that option and the preregistered experiments that shaped it. In SEMVM (Stateful Semantic Virtual Machine), a learned skill is a small typed program stored in a SQLite procedure library, with its source evidence, version history and verification record. The program is

not written by a language model. It is abstracted by a deterministic anti-unifier from traces that the system itself executed while a teacher demonstrated the skill in natural language, and it becomes executable only after a deterministic verifier has tested it against that evidence. The student’s neural weights never change; their byte-level hash is checked in every process.

We use the term *external procedural distillation* for this setting: a capability demonstrated by a larger model ends up as verified external memory of a smaller system. The term is deliberately narrow. The 1.5B model does not learn the 120B model’s knowledge; no weights are trained; nothing is distilled into parameters. What is transferred is a procedure, and what makes it trustworthy is not the teacher’s authority but the evidence and the verifier.

Three roles. It is easy to read “a small frozen model acquires capability” more generously than the evidence allows, so we state the roles plainly. *The library is the capability*: what the system can do after handoff is exactly the set of ACTIVE verified procedures. *The 120B model is the acquirer*: during acquisition it is the semantic engine, writing the lessons and grounding each lesson line into a typed plan. *The 1.5B model is a fixed retrieval front end*: after handoff it ranks ACTIVE procedure names and writes argument text, both checked deterministically. The 1.5B model does not interpret lessons in the final configuration, and nothing here shows that it could.

Core claim. The SEMVM system can accumulate externally stored procedural capability, acquired with assistance from a temporary teacher, without modifying the frozen student’s neural weights. Continued growth of that library currently depends on access to a teacher or other source capable of supplying usable demonstrations. The teacher sits in the *acquisition* causal chain; after handoff it is absent from the *execution* causal chain (Figure 3).

What we do not claim. We do not claim that the 1.5B model learned the 120B model’s knowledge; classical (weight-level) distillation; open-domain continual learning; autonomous self-improvement; arbitrary program synthesis; indefinite scaling of the library; or human-level generality. We do not claim that the final LOCKED evaluation was a formally clean pass: it was not (Section 6).

Contributions.

1. An architecture in which neural components (teacher, grounder, request interpreter) have proposal authority only and never hold state: the world is read through typed queries executed by a deterministic runtime, which holds all decision authority over state, evidence admission and procedure activation (Section 3, Tables 1 and 2).
2. A transactional teaching protocol: a demonstration is a sandboxed transaction over a slot ledger that either commits complete evidence or aborts with zero evidence (Figure 2).
3. A preregistered lineage of seven experiments, including negative and invalid runs, that locates the bottleneck step by step: runtime, procedure induction, natural-language grounding, teacher channel, clarification protocol, transactional safety, and finally recovery capacity (Section 4, Figure 4).
4. A 2×2 factorial over lesson source and recovery source, replicated on two independent development draws, showing that once resources suffice these factors move acquisition by at most a few targets and in inconsistent directions (Section 5).
5. A fully disclosed LOCKED result whose behavioral criteria all pass but whose preregistered teardown-proof condition fails, reported as such, with an audit of every gate-deciding check that treats the failure as a verification-of-verification finding (Section 6), and a released artifact bundle with raw teacher, grounder and verifier records (Section 12).

2 Related work

Knowledge distillation and continual learning. Distillation transfers a teacher’s behavior into student weights through soft targets [11], sequence-level outputs [13] or rationales [12]. Continual learning studies

how to add capability to weights without catastrophic interference [14, 19]. SEMVM sidesteps both by never training the student: the transferred object is an external program, so interference with existing weights cannot occur, but neither does the student’s general competence improve.

Retrieval and external memory. Retrieval-augmented generation conditions a model on retrieved text [16]; memory-management systems page information in and out of a context window [20]; differentiable memories make storage part of the network [9]. In these systems the retrieved content is read by the model and influences generation. In SEMVM the stored object is executed by a deterministic VM; the language model at reuse time only ranks candidate procedure names and writes an argument list, both of which are checked (Section 3). A wrong retrieval can cause a clarification or an incorrect choice of procedure, but it cannot alter what a stored procedure does.

Tool making and skill libraries for LLM agents. LLMs can write reusable tools [4, 30], accumulate skill code in open-ended environments [27], induce reusable workflows from experience [28], or store textual insights [26, 32]; tool-use models call APIs [21, 24, 29], and program-aided models offload computation to an interpreter [7]. These systems typically let the LLM author the stored artifact and validate it by task success or execution feedback. SEMVM differs in where authority sits: the stored program is derived from executed, committed traces by a deterministic abstraction step, and its activation is decided by evidence tests that include counterfactual and perturbation checks designed to catch programs that are consistent with the demonstrations yet wrong (Section 3.5). In our own lineage, the 1.5B model’s direct program proposals added nothing over anti-unification (Section 4).

Library learning and program induction. DreamCoder [6] and LILO [8] compress solved programs into reusable abstractions; Berlot-Attwell et al. [3] caution that measured gains in LLM library learning can come from other factors than reuse. Our abstraction step is anti-unification-style generalization [22] over a small typed step language, closer in spirit to programming-by-example systems such as FlashFill [10]. We make no claim about library-learning at scale; our reuse measurements are direct (the stored procedure is invoked after restarts with new arguments and wording).

Learning procedures from instruction and demonstration. Programming by demonstration [5], instructable agents that learn procedures from natural-language dialogue [2], multimodal demonstration systems [17] and collaborative task learning from a single session of demonstration and dialogue [1] are the closest ancestors of the teaching setting. Our teacher is a model rather than a person, the student is a frozen small model plus a deterministic runtime, and the protocol is evaluated with preregistered gates and sealed test sets.

Semantic parsing and neural program execution. Grounding language into executable forms is classical semantic parsing [18, 31]; dataflow dialogue [25] is a close relative of our reference handling. Neural program interpreters [23] learn to execute programs in weights, whereas SEMVM keeps execution symbolic. Compositional generalization failures of sequence models [15] motivated our earlier choice to place state and bookkeeping outside the network.

3 System

3.1 Overview and authority boundaries

Figure 1 shows the final configuration and Table 1 lists each component with what it may and may not do. The organizing rule is that neural components produce *proposals* (lesson text, a typed plan for one line, a ranking of procedure names, an argument string), and deterministic components decide whether anything is executed, admitted as evidence, or activated.

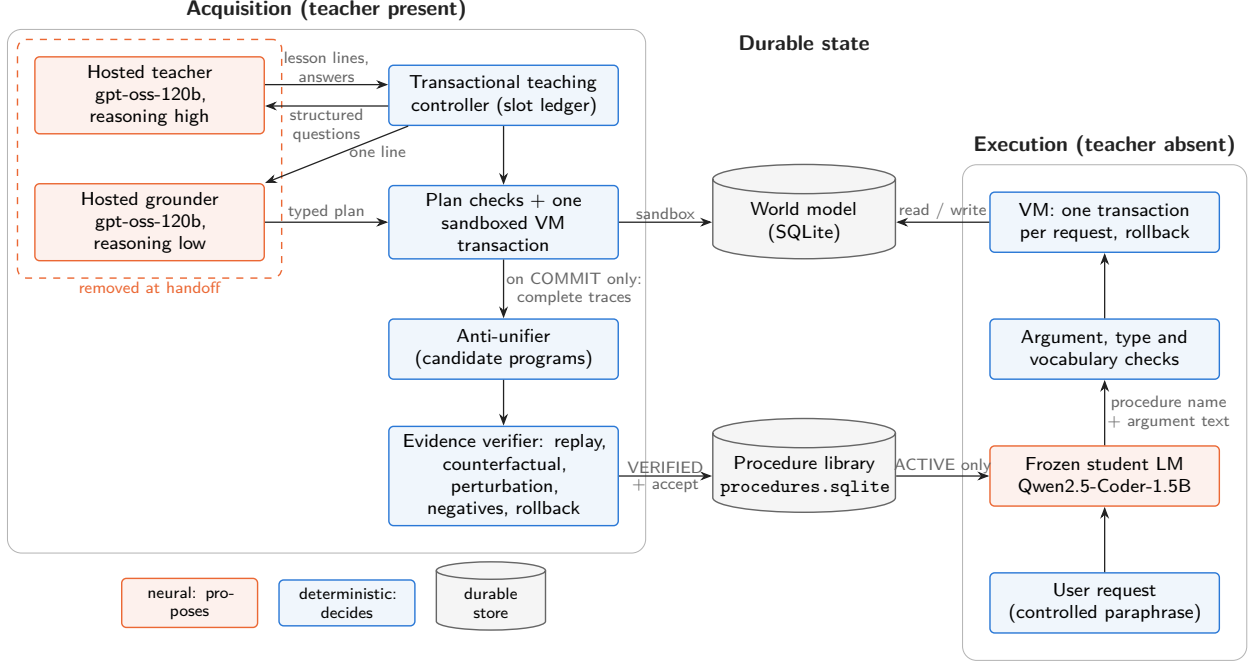


Figure 1: SEMVM architecture in the final (RCI) configuration. Orange components are neural and only propose; blue components are deterministic and decide; grey cylinders are durable stores. The two hosted roles share one frozen gpt-oss-120b checkpoint and are destroyed at handoff. After handoff, requests are interpreted by the frozen 1.5B model and executed only through ACTIVE procedures.

3.2 State lives outside the context window

A conventional LLM agent is monolithic in where it keeps state: the conversation, retrieved documents, tool outputs and intermediate results accumulate in the context window, learned behavior lives in the weights, and the model itself is the only component that reads that state and decides what it means. SEMVM inverts this (Table 2). The world model and the procedure library are the only durable state. No model ever receives the world as text: state is read and written only through typed queries and transactions that the VM executes (FIND, GET, CHILD, SET_STATUS, ...), and each neural call receives a small, purpose-built view and returns a typed proposal. In the final configuration:

- the **teacher** sees the skill’s goal text, its example-value pool, the names and signatures of saved skills, and its own dialogue for the current target; it never sees world rows (median 1,310 input tokens over the 212 cached lesson calls of RCI);
- the **grounder** sees one instruction, the typed list of results defined so far in the current demonstration (e.g. “r2: r1 sorted by when (1 element)”), the primitive inventory and ACTIVE signatures, with no dialogue history (median 4,488 input tokens over 768 calls, most of it the fixed card and few-shot examples);
- the **student model at reuse** sees a fixed few-shot block, the signatures (name, typed parameters, return type) of ACTIVE procedures and the request, and ranks names or writes an argument list; it never sees the procedures’ bodies or the world.

Because the context carries references (r1, request:000011) rather than contents, its size depends on the instruction and the protocol, not on how large the world is or how many turns have passed; what the model “knows” about the world at any step is exactly what a query returned. Every answer can therefore be traced to specific rows, transactions and procedure versions, a model cannot silently alter state, and nothing depends on a transcript surviving a restart—the property that the handoff exploits. Earlier experiments in the project tested this placement directly at larger scale (Appendix J): a store scaled from 1K to 1M records at a constant working set, a 64-turn closed loop at about 24 model-facing tokens per turn against a

Table 1: Components and authority boundaries (final configuration).

Component	Implementation	May	May not
Hosted teacher	gpt-oss-120b, reasoning high, $T=0$, max 8,192 tokens	write EXAMPLE and STEP lines; answer structured questions	execute anything; see the world state; write the library; survive handoff
Hosted grounder	same checkpoint, reasoning low, separate role and cache	map one lesson line (plus the live result list) to a typed plan in JSON	choose among ambiguous referents; emit an unlicensed terminal action; execute
Teaching controller	deterministic state machine + slot ledger	order slots, ask questions, commit or abort a demonstration	admit an incomplete or aborted demonstration as evidence
Plan checks + VM	deterministic; one SQLite transaction; rollback	validate ops, types, references; execute in the sandbox	run code; execute an unvalidated plan
Anti-unifier	deterministic	propose candidate programs from committed traces	propose from uncommitted material or from gold
Evidence verifier	deterministic test battery	mark a candidate VERIFIED or REJECTED	consult gold programs or the teacher
Procedure library	<code>procedures.</code> <code>sqlite</code>	store versions, lifecycle, provenance, tests	activate without VERIFIED + explicit accept
Frozen student LM	Qwen2.5-Coder- 1.5B, frozen	rank ACTIVE procedure names; write argument text	bypass argument, type and vocabulary checks; create or modify procedures
Judge (evaluator only)	gpt-oss-120b, separate role	label lesson faithfulness for reporting	influence any system decision

transcript that grew to 1,306, and a frozen backbone reading a 512-entity store through typed operations at 29.2 tokens per turn, where a context-stuffing baseline needed about 20,000 tokens and failed (a comparison its report notes is confounded with out-of-distribution length). Those results are background, not evidence released here. In the bounded worlds of this paper the relevant observation is architectural: none of the prompts above contains world state by construction.

3.3 World model and VM

The domain is a personal organizer: events REMINDER, REQUEST, PLAN and NOTE, each optionally about an item CALL, EMAIL, VISIT or MESSAGE; relations WHO, RECIPIENT, WHEN, WHERE and TOPIC; and an open/closed status. The world is a SQLite store with persistent identifiers (`request:000011`), canonical state hashing and an audit table. Every request or teaching slot executes as a transaction with postconditions and rollback on failure; the VM executes a frozen inventory of 21 typed primitives (e.g. FIND, FILTER, SORT, FIRST, COUNT, SET_STATUS, FOR, CALL, RETURN) and no arbitrary code. This runtime was established in the first two experiments of the lineage and carried forward hash-checked thereafter.

3.4 Procedures and their lifecycle

A procedure is a typed program over the primitives with named parameters (Figure 7, step 7). Its identity is the hash of its canonical AST. The library stores every version with lifecycle state CANDIDATE $\rightarrow$ TESTING $\rightarrow$ VERIFIED $\rightarrow$ ACTIVE (or REJECTED_*), its source interactions and trace hashes, the proposer, and every test result; an `active_pointer` table names the one executable version (Figure 2b). Rejected versions are retained but can never be executed.

Table 2: Where state lives: a monolithic LLM agent versus SEMVM.

	Monolithic LLM agent	SEMVM
Working state	tokens in the context window (conversation, tool outputs, retrieved text)	rows in a SQLite world model, read through typed queries; context holds references
Learned skills	weights (fine-tuning) or prompt text	versioned, verified programs in procedures.sqlite
Who executes	the model, token by token	a deterministic VM, one transaction per request
Who decides what changes	the model	deterministic checks, verifier and explicit accept
Growth with history	context grows with turns and retrieved content	context bounded by protocol; state grows on disk
After restart	state must be re-supplied in context	state and skills persist; the model is reloaded unchanged
Audit	inspect a transcript	state hashes, transaction diffs, test records, provenance

3.5 Evidence verifier

Candidates are judged only against evidence from committed traces, never against gold. The battery (Table 7) consists of static validation; interface checks; replay of each source trace; counterfactual argument substitution, where the reference is the *unabstracted* trace with values substituted; state perturbations (e.g. all events closed, alternate events closed) that expose dropped constant filters; negative cases (missing input, wrong type, empty result, ambiguous entity, deleted object, empty world); rollback injection after every state-changing step; and determinism. The perturbation stage was added before any development run of the second experiment, after the unit suite showed that a verifier without it accepted a candidate that dropped a constant `STATUS=OPEN` filter because every evidence state happened to have all events open. The general lesson is that an evidence-only verifier can only be as discriminating as the diversity of the states it is allowed to test in.

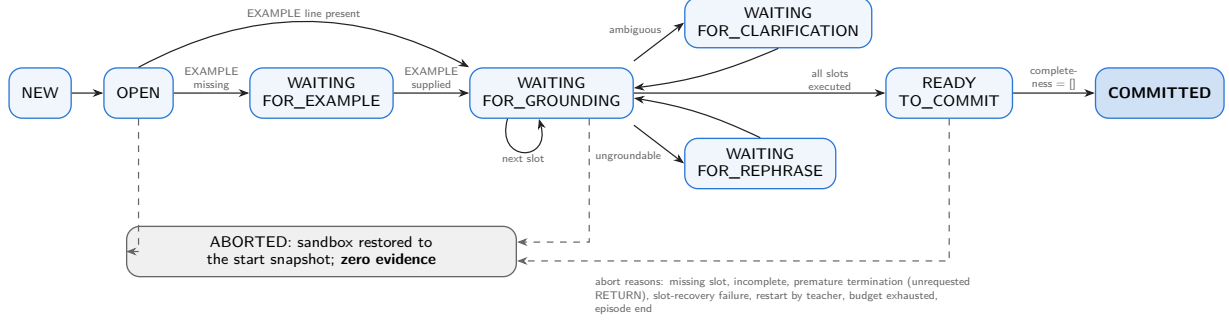
3.6 Transactional teaching

A lesson is an `EXAMPLE` line giving the example parameter values, followed by `STEP` lines. The controller turns the lesson into a slot ledger and runs the state machine of Figure 2a. Its registered rules include: an `EXAMPLE` is required before any step executes; a terminal `RETURN` is licensed only on the last slot and only when the goal asks for an output; saved skills are called only by name with explicit inputs; if more than one live result could fill a reference, the student asks a structured question listing the legal referents and accepts only `REFERENT: <id>`; grammar-legality feedback (e.g. an inner event’s time is `PARENT.WHEN`) is returned on rejection; lines injected by the evaluator are never presented to the teacher as the teacher’s own; and a demonstration commits only if the completeness check finds no unexecuted slot. An abort restores the sandbox snapshot and contributes zero evidence. Budgets are fixed per target (Appendix B).

3.7 Handoff and reuse

After acquisition the hosted app is stopped, the endpoint is probed (the registered requirement is `HTTP 404 / unavailable`), lesson transcripts and teaching context are removed from the student’s directory, teacher credentials are scrubbed from the environment, and every subsequent reuse process runs under an in-process network guard that patches Python’s socket connection calls to block and log non-loopback connections (Figure 3). The guard is instrumentation rather than isolation (Section 6.1), and the paper’s “zero hosted calls” and “zero network attempts” are the same measurement: the count of connections it logged. Reuse requests use wording that did not appear in the lesson and, where registered, arguments never demonstrated, and are interleaved with process restarts. A request is served by (i) a deterministic alias rule or (ii) the frozen 1.5B model ranking the `ACTIVE` procedure names by log-probability with a registered margin of 1.0 nats,

(a) One demonstration = one transaction (teaching controller)



(b) Procedure lifecycle (library; every transition logged in lib_log)

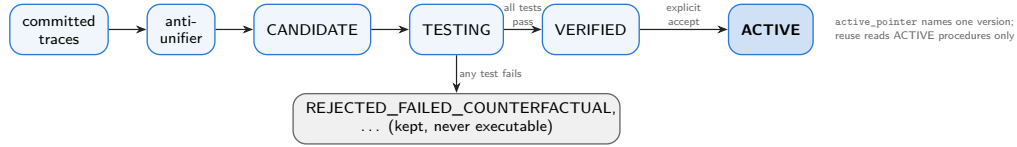


Figure 2: (a) A demonstration is one transaction: only COMMITTED demonstrations become learning evidence; any abort restores the sandbox and yields zero evidence. (b) Procedure lifecycle in the library: activation requires every verifier test to pass and an explicit accept.

below which the system clarifies; the model then writes the argument list, which must pass type, vocabulary and presence checks, with a typed fallback; otherwise the system clarifies.

3.8 Evaluation vocabulary

All metrics are computed by an evaluator that uses an independent oracle interpreter. The primary metrics are: **learned procedure exact** (the ACTIVE program for a target has the same canonical hash as the registered reference program; behaviorally equivalent variants count as failures); **scenario exact** (every acquisition and post-handoff turn of the scenario matches the oracle, including world state); **distilled skill exact** (a target is acquired, verified, activated, the teacher is verifiably destroyed, and every post-handoff invocation is exact with zero hosted calls); and **grounding behavioral exact** (a grounded demonstration is behaviorally equivalent to the reference for its own example values across five registered worlds). Safety metrics count wrong committed actions, incomplete or aborted evidence committed, unrequested terminal actions executed, ambiguous references executed and bad procedures activated. Gates were registered before each run; we report numerators and denominators throughout.

4 Research progression

Figure 4 and Table 3 summarize the seven experiments. Each was specified in a preregistration document with gates, stop rules and a sealed LOCKED set; each stopped at its registered decision point. About six weeks of earlier preregistered experiments (Appendix J; code and results not part of this release) are not part of this paper’s evidence; their main influence was a repeated observation that multi-step bookkeeping fails when left inside a network and becomes exact when made explicit in a deterministic runtime, and that a small controller can act over a large store at constant context when it reaches state through typed queries.

E2E (END_TO_END_POC_V1). A small frozen neural binder from earlier work (checkpoint hash-pinned) translated a controlled compositional language with multiple modifier bindings into a canonical intermediate representation, which a deterministic resolver and VM applied to the SQLite world. RESULT A within the registered regime: LOCKED 20/20 scenarios exact, DEV 29/30 (the single miss was one neural parent-binding error), and the gold-IR control exact on both suites. The input language was a controlled,

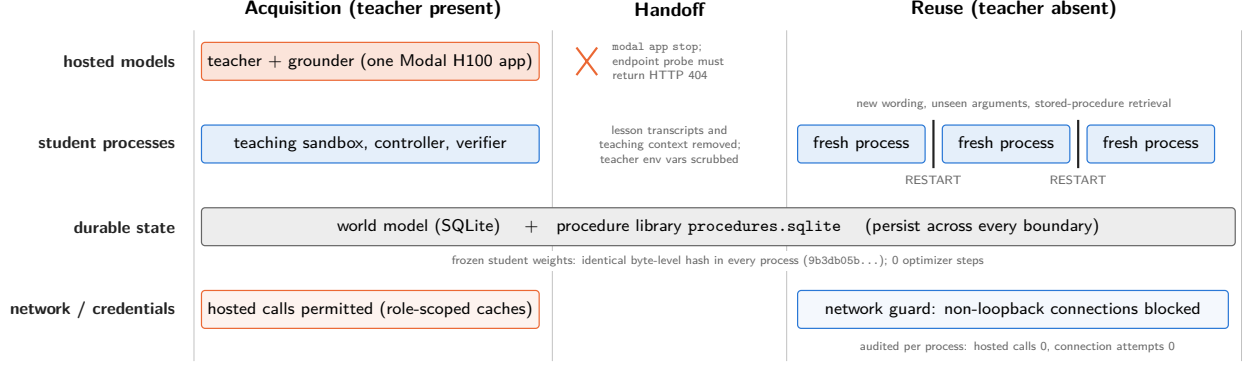


Figure 3: Teacher removal and persistence. Only the world model and procedure library cross the handoff; the frozen student’s weight hash is checked in every process. The LOCKED run’s handoff probe returned HTTP 401 rather than 404 (Section 6).

tag-coindexed language; English was out of scope and rejected (0/8 exact, 8/8 rejected by the envelope check).

PDISC (PROCEDURE_DISCOVERY_POC_V1). Added the procedure library, deterministic anti-unification and the evidence verifier, with teaching in a formal step language and retrieval by controlled paraphrase. RESULT A: LOCKED_v2 20/20 with every registered gate at 1.0, and 0 of 6 deliberately bad candidates accepted. The frozen 1.5B model was also asked to propose procedures: every procedure it got right, anti-unification also got right (0 LLM-only verified discoveries of 32 on LOCKED). It was load-bearing for retrieval ranking, not for induction. A first LOCKED set was invalidated before being run after a scenario-design defect was found on DEV, and a fresh one was sealed.

NL_TEACH. Replaced formal teaching with natural-language lessons grounded by the frozen 1.5B model through grammar-constrained ranking of legal typed options and a deterministic multi-clause plan compiler. RESULT B with a safety stop after three registered DEV iterations: learned .914, scenario .867, 1 wrong action executed (22 wrong raw proposals, 16 blocked or repaired), and 1 verifier false accept caused upstream by a lost second demonstration. The strongest observation was methodological: the same model writing the program directly from the lesson was exact for 5.6% of procedures, while trace-mediated learning was exact for 86.1%. LOCKED was not run.

PDX (hosted teacher). Introduced a hosted gpt-oss-120b teacher and the handoff protocol. Retention was positive: every exactly acquired skill was reused exactly after the teacher was destroyed (54/54 invocations). Acquisition was negative: the hosted teacher reached 24/36 learned procedures against 33/36 from fixed scripted lessons, with 12 wrong executed demonstrations, split roughly evenly between teacher-side errors (unrequested constraints) and student-side mis-grounding. Two DEV attempts were superseded before the final one (a scenario-design defect; truncation of 32/98 teacher replies at a 1,200-token cap). LOCKED was not run.

HTG (hosted teacher × grounder). A 2 × 2 of teacher (scripted vs hosted) and grounder (local 1.5B vs hosted 120B), with added evidence non-vacuity and conflict checks. Arm D (hosted × hosted) had grounding behavioral exact 1.0 but learned only 27/36 and executed 2 wrong actions. Both wrong actions share a lesson that we carry forward: *consistent evidence is not necessarily complete evidence*. In one, an unrequested RETURN ended a demonstration after one step and a one-step wrapper verified against that truncated evidence; in the other, an injected vague step was never restored and the resulting program returned calls instead of people. The verifier behaved as specified in both cases. The bottleneck had moved from grounding accuracy to the clarification protocol. LOCKED was not run.

experiment	added	observed (registered outcome)	remaining limitation → next question
E2E end-to-end POC	frozen neural binder + SQLite world model + VM; controlled input language	RESULT A: LOCKED 20/20, DEV 29/30; runtime exact given correct semantics	no procedures; English outside the envelope
PDISC procedure discovery	procedure library, anti-unifier, evidence verifier; formal step-language teaching	RESULT A: LOCKED_v2 20/20; 0 false accepts; LLM-only discoveries 0 of 32	teaching is formal, not natural language
NL_TEACH natural-language lessons	frozen 1.5B grounds each lesson line; deterministic plan compiler	RESULT B + safety stop: learned .914, wrong executed 1; trace-mediated 86.1% vs direct 5.6%	multi-clause grounding by the 1.5B model
PDX hosted teacher	gpt-oss-120b teacher; verified teardown; post-handoff reuse	retention 54/54 given exact acquisition; teacher 24/36 vs scripted 33/36; wrong 12	teaching channel (lesson + grounding)
HTG teacher × grounder	hosted 120B grounder; 2×2 factorial; non-vacuity and conflict checks	RESULT G+E: D learned 27/36, grounding 1.0, wrong 2	consistent evidence need not be complete: clarification protocol
TTP transactional teaching	slot ledger, commit/abort, structured referent choice, completeness before commit	RESULT D: D learned 31/36, wrong 0, completeness 1.0	recovery capacity: value pool, truncation, budgets
RCI recovery capacity	value pool 2→4; teacher max_tokens 4096→8192; two DEV draws	D pooled learned 70/72; LOCKED 23/23 behaviorally, teardown proof failed (401)	library growth still needs a teacher

Figure 4: Research progression. Each experiment added one layer and was stopped at its registered decision point; the residual limitation of each became the next question.

TTP (transactional teaching). Treated a demonstration as a transaction (Figure 2a) and fixed each HTG failure class in general form. Arm D passed every safety gate (0 wrong actions, 0 incomplete commits, completeness 1.0, 5 unrequested RETURNS blocked) but not accuracy: learned 31/36 (.861), scenario .833. The five misses were attributed to provider truncation at 4,096 tokens, a two-value teacher pool that could not survive a rejected legal-bad demonstration, a per-target turn budget, and one false clarification (RESULT D). The first FINAL_DEV attempt was invalid and archived (Section 8). LOCKED was not run.

RCI (recovery capacity isolation). The centerpiece, described next.

5 Recovery capacity isolation

5.1 Design

RCI asked whether TTP’s residual failures required a new recovery architecture or only more resources. The controller, prompts, grounder, verifier, budgets and thresholds were inherited unchanged, with TTP’s post-run Amendment 003 (count committed demonstrations, not indices) as the baseline rather than a treatment.

Stage 0. (A) A parity audit using only generator source and manifest metadata found that the parent’s sealed LOCKED set embedded a two-value teacher pool at generation time, so a successor LOCKED set was generated from the same never-inspected blind templates with a new seed and sealed before any RCI development run. (B) A failure taxonomy with an earliest-cause rule and fixed precedence (teacher → grounder → controller → evidence → procedure) was frozen before scoring. (C) A non-confirmatory ceiling diagnostic re-ran the five parent Arm D failures with relaxed budgets; 4/5 recovered exactly, which was the registered go signal.

Table 3: Experiment lineage. Outcome labels are those registered in each experiment’s own preregistration. “LOCKED” is the sealed test set; “not run” means the registered stop rule fired on DEV and the sealed set stayed unopened.

Experiment	Question	Key observed numbers	LOCKED
E2E	Does a frozen binder + deterministic VM form a working persistent runtime?	DEV scenarios 29/30; binder errors only (1 parent-binding error)	run once: 20/20
PDISC	Can procedures be induced from traces and reused after restart?	0 false accepts on bad candidates; LLM-only discoveries 0 of 32	run once: 20/20
NL_TEACH	Can a frozen 1.5B model ground natural-language lessons?	learned .914; scenario .867; wrong executed 1; false accept 1; trace-mediated 86.1% vs direct 5.6%	not run
PDX	Can a hosted 120B teacher teach the frozen student?	teacher learned 24/36 vs scripted 33/36; wrong 12; false accept 1; retention 54/54	not run
HTG	Does a hosted grounder fix the teaching channel?	Arm D learned 27/36; grounding 1.0; wrong 2	not run
TTP	Does a transactional protocol remove unsafe acquisition?	Arm D learned 31/36; wrong 0; completeness 1.0	not run
RCI	Is the residual recovery capacity?	Arm D pooled learned 70/72; scenario 58/60; wrong 0	run once: 23/23 learned; not a clean pass

Stage 1 intervention. Exactly two settings changed: the teacher example-value pool from 2 to 4 values per class (each with witness events), and the teacher `max_tokens` from 4,096 to 8,192. The serving context length was raised from 16,384 to 32,768 only so that these budgets fit; it is a serving parameter with the same checkpoint and sampling. The configuration fingerprint (`dacd7a90...`) was frozen before the first development draw and is identical in both draws and in LOCKED.

Factorial. All four arms used the same hosted grounder. Lesson source was scripted (registered lessons) or hosted (the teacher); recovery source (the answers to rephrase and reference questions) was scripted (a frozen lookup of registered rephrasings), hosted (a separately cached recovery-teacher role), or oracle (answers derived from the registered reference program, and only when the executed prefix matched it). Arm D (hosted lesson, hosted recovery) was the confirmatory arm; B, E and F were attribution arms. Two development draws (DEV-A, DEV-B; 30 scenarios and 36 targets each; 20 strata) were run with no change of any kind between them. Primary gates were pooled with per-replicate floors of .80, and safety gates had to hold in each replicate independently.

5.2 Results

Arm D passes every pooled gate. Pooled over both draws, Arm D learned 70/72 procedures exactly (registered minimum 65), with 58/60 exact scenarios (minimum 54), 62/64 distilled skills (minimum 58) and 71/71 grounding (minimum 68); per replicate the values were 36/36 and 34/36 learned (Table 4). In each replicate there were zero wrong committed actions, zero incomplete commits, zero unrequested or ambiguous executions and zero bad activations; replay, counterfactual, rollback and reuse given acquisition were all 1.0; the teardown probe returned 404 in both draws. For orientation only, descriptive 95% Wilson intervals are [.904, .992] for pooled learned (70/72) and [.886, .991] for pooled scenario (58/60); no significance test is implied. Hosted-teacher truncations fell from 9 in the parent Arm D to 0, and no pool-exhaustion failure occurred.

Table 4: Parent (TTP) versus successor (RCI) for Arm D. The parent row is its own DEV draw; the RCI rows are two further independent draws. Draws differ, so the comparison is indicative, not paired.

Metric (Arm D)	TTP parent	RCI pooled	RCI DEV-A / DEV-B
Learned procedure exact	31/36 (.861)	70/72 (.972)	36/36 / 34/36
Scenario exact	25/30 (.833)	58/60 (.967)	30/30 / 28/30
Distilled skill exact	27/32 (.844)	62/64 (.969)	32/32 / 30/32
Grounding behavioral exact	32/38 (.842)	71/71 (1.0)	36/36 / 35/35
Wrong committed actions	0	0	0 / 0

Table 5: RCI factorial, all arms with the hosted 120B grounder. Cells are DEV-A + DEV-B = pooled. The grounding column uses the registered gate definition, which also counts faithful demonstrations that were later aborted.

Arm	Lesson / recovery	Learned	Scenario	Distilled	Grounding
B	scripted / scripted	36/36 + 35/36 = 71/72	30/30 + 29/30 = 59/60	32/32 + 31/32 = 63/64	36/37 + 35/41 = 71/78
E	scripted / hosted	36/36 + 36/36 = 72/72	30/30 + 30/30 = 60/60	32/32 + 32/32 = 64/64	36/36 + 36/36 = 72/72
F	hosted / oracle	34/36 + 34/36 = 68/72	28/30 + 28/30 = 56/60	30/32 + 30/32 = 60/64	35/35 + 37/38 = 72/73
D	hosted / hosted	36/36 + 34/36 = 70/72	30/30 + 28/30 = 58/60	32/32 + 30/32 = 62/64	36/36 + 35/35 = 71/71

Residual failures. DEV-A had none. DEV-B had one `TEACHER_INITIAL_SEMANTIC_ERROR` and one `EVIDENCE_NONVACUITY_FAILURE`. No cause repeated across draws, and the registered trigger for a Stage-2 structured-clarification experiment (mass on grounder false clarification or recovery semantic errors) did not fire.

Factorial interpretation. Pooled learned counts were B 71/72, E 72/72, F 68/72 and D 70/72 (Table 5, Figure 5). We do not rank the arms. Three observations are nonetheless supported by the records:

- *Scripted recovery, not hosted recovery, was the weak recovery source.* Arm B’s scripted recovery replays a registered rephrasing, which the hosted grounder often clarified again: 0 of 14 recovery slots in B grounded, and B recovered by starting new demonstrations instead. The hosted recovery teacher in E resolved 12 of 12 recovery events.
- *The hosted initial lesson costs a little.* F and D lost two to four targets to initial-lesson content errors, one per replicate; lesson faithfulness as labeled by the judge was .92–.97.
- *The contrasts are small and their signs do not replicate.* For example, D–E on learned was 0 on DEV-A and $-.056$ on DEV-B; the interaction term was $+.056$ and $-.028$. With adequate resources, lesson source and recovery source each moved acquisition by at most about two of 36 targets.

The grounding column is lower for B (71/78) because the registered grounding metric counts faithful demonstrations that reached grounding even if they were later aborted. Committed demonstrations were behaviorally exact in every arm and replicate except one in F on DEV-B (44 of 45), the single wrong committed action of the factorial, attributed to a hosted initial-lesson semantic error under oracle recovery.

6 LOCKED evaluation

The LOCKED scope and pass rule were registered after the pooled decision and before the run: one run of the 20 sealed successor scenarios (23 targets), arms `GOLD_TRACE`, `FORMAL_TEACH` and D, the identical frozen configuration, and the same thresholds as the pooled gates, with no rerun under any outcome.

The sealed LOCKED evaluation passed every behavioral acquisition, grounding, safety, and reuse criterion, but failed the preregistered proof-of-teacher-disconnection condition because teardown was probed before the hosted endpoint had fully disappeared.

Table 6 lists the criteria as recorded and under the strict registered rule. After LOCKED acquisition, `modal app stop --yes` returned 0, but the immediate endpoint probe returned HTTP 401: the app was still draining and rejecting unauthenticated requests. The evaluator’s teardown helper, inherited from HTG, accepted any HTTP error as “destroyed”, recorded the teardown as verified, and started reuse. Under the

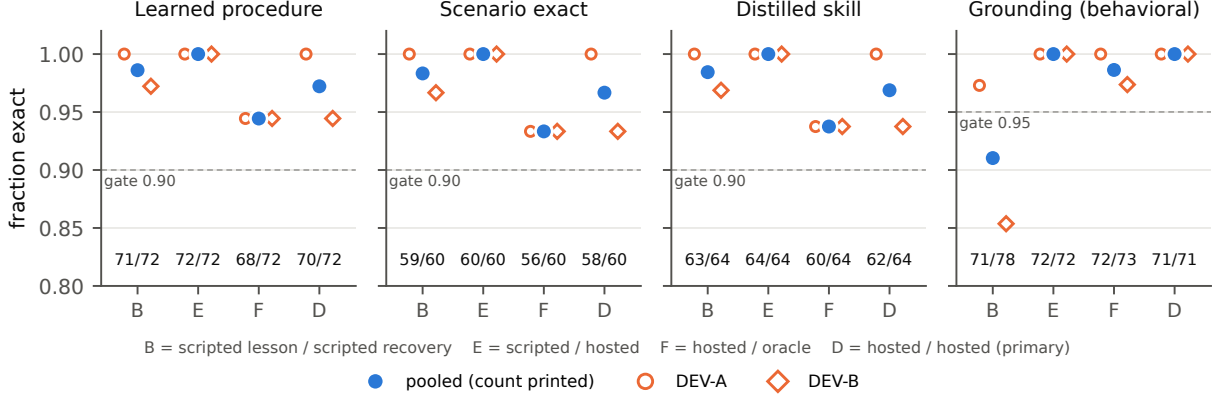


Figure 5: RCI factorial: pooled rates (filled) and per-replicate rates (hollow) with counts; dashed lines are Arm D’s registered gates. The vertical axis starts at 0.8; all differences between arms are a few targets.

Table 6: LOCKED (20 sealed scenarios, 23 targets, Arm D). “Strict” applies the registered teardown rule (HTTP 404 required).

Criterion (Arm D, LOCKED, 20 scenarios)	As recorded	Strict (registered rule)	Gate
Learned procedure exact	23/23	23/23	$\geq .90$
Scenario exact	20/20	20/20	$\geq .90$
Grounding behavioral exact	22/23 (.957)	22/23	$\geq .95$
Wrong committed actions	0	0	$= 0$
Incomplete demonstrations committed	0	0	$= 0$
Post-handoff hosted calls / network attempts	0 / 0	0 / 0	$= 0$
Unseen-arg. / restart / stored reuse given acquisition	1.0 / 1.0 / 1.0	1.0 / 1.0 / 1.0	$= 1$
Hosted teardown verified	1.0 (probe HTTP 401 accepted)	0 (401 $\neq$ 404)	$= 1$
Teacher-disconnect reuse given acquisition	1.0	0.0	$= 1$
Distilled skill exact	21/21	0.0	$\geq .90$

registered rule the teardown-verification gate is 0, and so are the two criteria that depend on it (teacher-disconnect reuse and distilled skill exact). The evidence that the student did not depend on the teacher is strong but is not the registered proof: every reuse process ran with teacher credentials absent and under the network guard, with 0 connection attempts logged by the guard; a later probe returned 404 and the app was listed as stopped; and all reuse results matched the oracle. The same helper had received a real 404 in both development draws and in every earlier experiment’s handoff. The helper has since been changed to poll until a real 404 and to reject 401, 403 and 5xx responses, although it still treats a non-HTTP failure of the probe itself as “unavailable” (Section 6.1); no RCI run used the changed version. LOCKED is not rerun. The two development draws, whose handoffs returned a real 404, are therefore what carry the causal claim; LOCKED is a behaviorally successful but formally incomplete confirmation.

6.1 Verifying the verification code

The teardown race is an instance of the paper’s own central lesson one level up. HTG showed that *consistent evidence is not necessarily complete evidence*: a program can agree with every committed trace while the traces omit part of the lesson. The teardown helper had exactly that property. An HTTP error is consistent with “the endpoint is gone” but is not complete evidence of it, and the helper accepted any HTTP error, so a 401, 403 or 500 would all have passed. We designed a check to prove teacher independence and did not check the check.

We therefore audited every helper in the released code that decides a registered gate, asking what it accepts as a pass and whether any released run passed through a permissive path (`docs/verification_audit.md`; read-only, no run repeated). Findings:

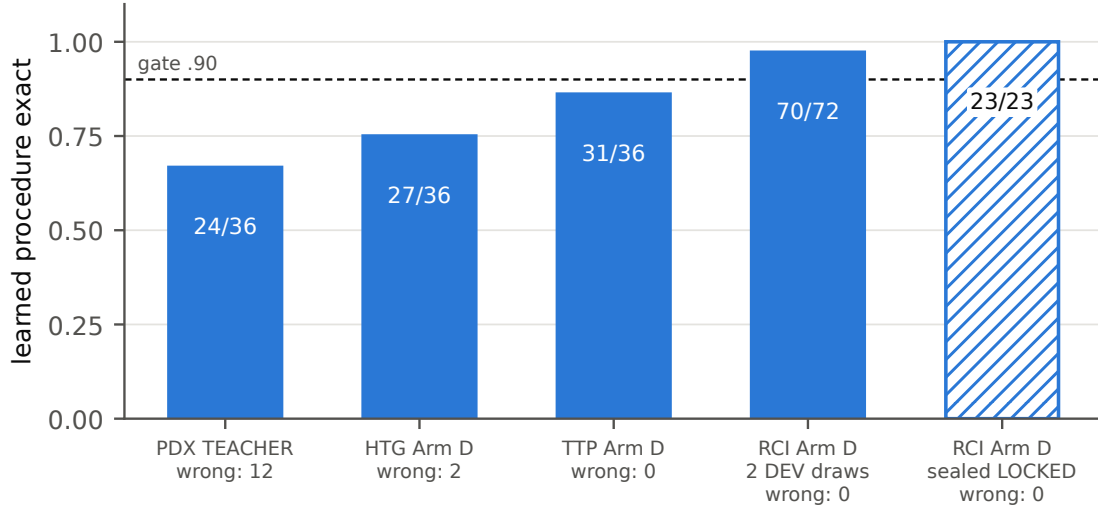


Figure 6: Learned-procedure rate of the hosted-teacher arm across the lineage (PDX used the local 1.5B grounder; HTG, TTP and RCI used the hosted grounder). Each bar is a different DEV draw; LOCKED is hatched and, per Section 6, is not a clean pass.

- **The permissive teardown class occurs in two helpers.** PDX’s rule accepted anything other than a successful response, including no probe at all if the endpoint address was not configured; the HTG/TTP/RCI helper accepted any HTTP error. In the released runs the permissive path was taken once, at RCI LOCKED; every other handoff record reads HTTP 404. The post-fix helper still counts a network failure on the prober’s side as teardown; a strict check needs an affirmative signal (404, or a provider-reported stopped state).
- **Hosted calls and network attempts are one measurement**, both derived from the in-process guard, which patches two Python socket calls and does not cover other channels (for example native-code sockets or subprocesses that do not load it).
- **Two aggregate checks would pass vacuously on empty input:** the per-process “clean” audit (an *all* over audit records) and weight-hash invariance (one distinct hash among the recorded ones). Neither input was ever empty; every RCI scenario has audit records and exactly one weight hash (9b3db05b...) occurs across all suites and arms.
- **The oracle is independent in execution semantics but shares parsing and program hashing** with the runtime (`procedure/lang.py`).
- **Completeness is relative to the lesson, not the goal:** the controller can guarantee it executed every line the teacher wrote, not that the teacher wrote every step the goal requires.

None of these findings changes a released number. They change how the disconnection evidence should be described (as above), and they argue for treating verification code as a first-class object of verification, with its own negative tests (a teardown probe fed 401, 403, 5xx or a timeout must fail; an aggregate over zero records must fail).

7 Worked example: dev_a-022-S15

Figure 7 traces one Arm D target from DEV-A end to end; every quoted line is copied from the released evidence dump (`results/examples/dev_a-022-S15_D.md`, regenerated by `dump_case.py`). The skill is `close_oldest_open_request(person)`: take that person’s open requests, sort them by time, close the first, and return it.

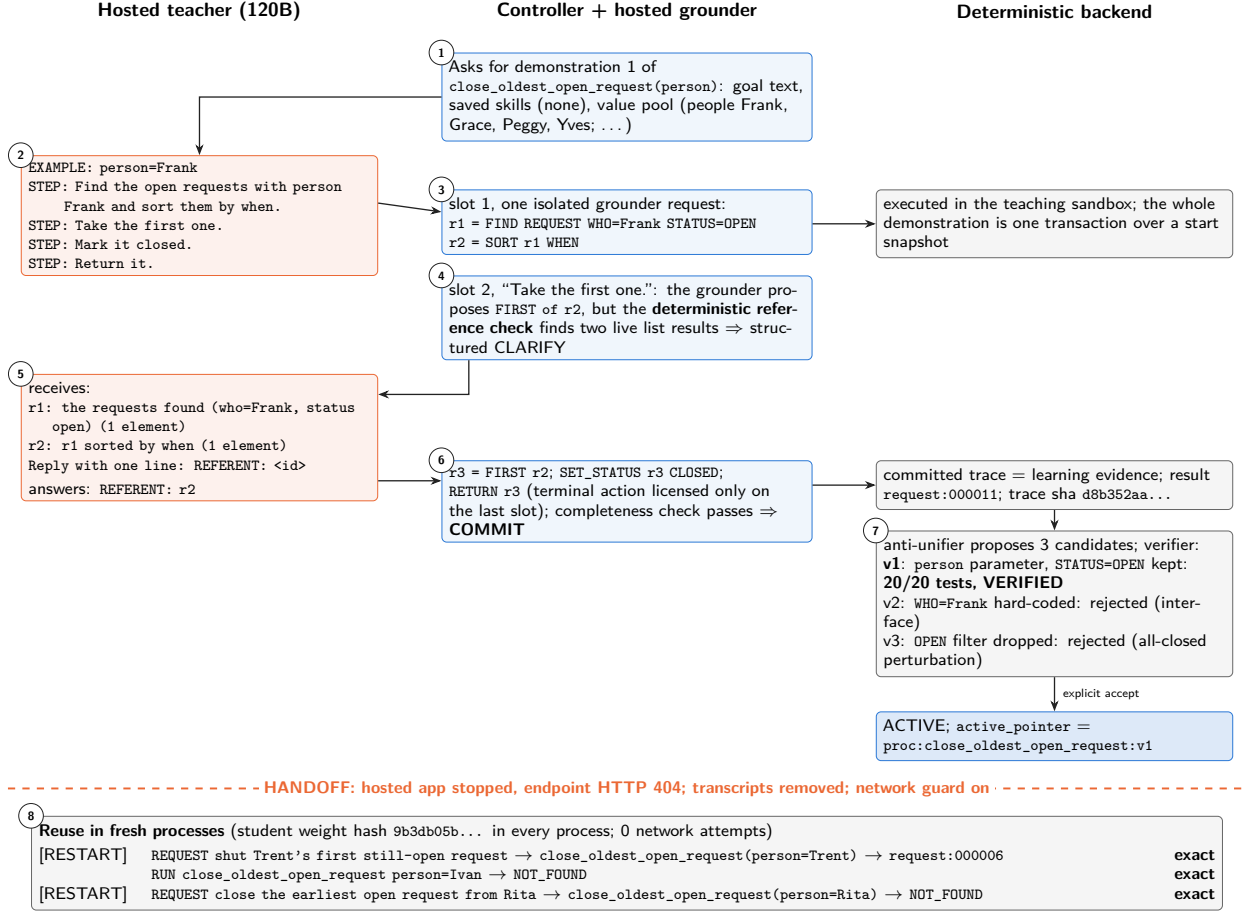


Figure 7: Worked example dev_a-022-S15, Arm D. Numbered steps follow the actual event order. The hosted grounder’s first reading of “Take the first one.” already pointed at `r2`; the controller clarified anyway because its deterministic reference rule saw two live list results. The anti-unifier produced three candidates; only `v1` survived verification.

Table 7: Verifier tests for the accepted candidate in the worked example.

Verifier test family	Passed
static validation	1/1
interface / signature	1/1
source replay	1/1
counterfactual argument substitution	2/2
state perturbation (all closed / alternate closed)	4/4
negative cases (missing input, wrong type, empty result, ambiguous entity, deleted object, empty world)	6/6
rollback injection after each state-changing step	4/4
determinism	1/1
Total	20/20

The teacher (1,285 input and 1,129 output tokens for the lesson) wrote a four-step lesson with **EXAMPLE: person=Frank**. The first line was grounded as a FIND followed by a SORT, producing two list results. For “Take the first one.” the controller’s reference rule found two live candidates, so it asked a structured question listing **r1** (the found requests) and **r2** (**r1** sorted by when), and the teacher answered **REFERENT: r2**. The remaining slots grounded directly, RETURN was licensed on the last slot, the completeness check passed, and the demonstration committed with result **request:000011**. From this single committed trace the anti-unifier proposed three candidates. The verifier accepted v1 (parameter **person**, **STATUS=OPEN** kept) with 20 of 20 tests (Table 7); it rejected v2, which hard-coded **WHO=Frank**, at the interface check, and v3, which dropped the **OPEN** filter, at the all-closed perturbation. The two registered false-accept probes (frozen value, dropped constant filter) were also rejected. After handoff, three post-restart requests in new wording were served from the library: Trent’s request was closed and returned exactly (**request:000006**), and the Ivan and Rita requests correctly returned **NOT_FOUND**.

Two caveats. First, Frank had only one open request, so in the demonstration the SORT and the choice of **r2** over **r1** made no observable difference; the clarification supplied identifying information (the intended referent) that behavioral evidence could not. The program’s use of SORT comes from the lesson text, not from anything the trace could have distinguished. Second, only Trent was a positive reuse case; Ivan and Rita returned **NOT_FOUND**. Whether the stored procedure picks the oldest of several open requests at reuse time is therefore untested by this example.

8 Integrity, invalid runs and amendments

We list every event that affected the validity or interpretation of a reported run. Full records are in each experiment’s engineering log and in Appendix H.

- **TTP attempt 1 was invalid.** The hosted grounder emitted a CHILD action without its required of field; it passed validation and crashed one Arm D acquisition. Per the registered engineering policy the whole run was archived as invalid, the defect was fixed in general form (a required-field check plus a no-crash guard; Amendment 002), and FINAL_DEV was rerun from scratch on a freshly seeded DEV set. The archived run’s summaries and diagnostic are released.
- **TTP Amendment 003.** After TTP’s valid run, a controller defect was found: the two-demonstration rule counted demonstration indices rather than committed demonstrations. In one scenario (**dev-019-S14L**) an aborted first demonstration left an evaluator-injected legal-bad demonstration to be learned alone, activating one legal-bad procedure in each of Arms A and C. No Arm D target was affected. The A/C safety results of that run are therefore not a valid measurement; the fix was unit-tested, not rerun within TTP, and became RCI’s baseline.
- **RCI Stage 0C teardown targeted the wrong app.** The ceiling-diagnostic script called the inherited teardown helper without setting its app name; it tried to stop the absent HTG app (return code 1), and the RCI endpoint remained reachable (401). This was repaired immediately by stopping

the correct app and verifying 404. Stage 0C was acquisition-only and non-confirmatory; no reuse ran under it.

- **The LOCKED 401 race** (Section 6).
- **Evaluator-only defects** found after runs and repaired with run data unchanged: in HTG, the judge process lacked credentials (labels regenerated later from saved lessons with a separately verified teardown) and a parameterless-reference scoring bug (one spurious wrong action per arm; the Arm D decision is unchanged either way).
- **Superseded development attempts** in PDISC (DEV v1, v2) and PDX (a scenario-design defect; teacher truncation at a 1,200-token cap). In PDISC, the first LOCKED set was archived unopened as invalidated-before-run.
- **Post-run code edits in the release.** Frozen-file hashes in the released tree match their freeze manifests except four documented post-run edits: TTP `teacher/transactional.py` and `tests/test_ttp.py` (Amendment 003), HTG `evaluator/behavior.py` (the evaluator repair) and RCI `htg_evaluate.py` (the post-RCI teardown fix). Path sanitization and one portability edit are recorded file-by-file with original and released hashes.

9 Discussion

External memory versus weights. Placing the learned skill outside the network made three things cheap that are hard in weight space: inspecting exactly what was learned (Figure 7, step 7), rejecting it before it can act, and keeping it through restarts without any interference with the model. The cost is scope: only what the step language can express can be learned, and the model’s own competence does not improve.

Query-mediated state versus monolithic context. The most consequential design choice is not the procedure library but the decision that no model holds state. Monolithic agents scale their memory by scaling context, retrieval or weights, and each of these puts the model in charge of reading and maintaining that state. In SEMVM the model’s job at every step is to translate one bounded input into one typed proposal; persistence, identity, bookkeeping and execution are properties of the runtime. This is what makes the teacher removable: nothing the student needs after handoff lives in a transcript or a context window. It is also a limitation: whatever cannot be expressed as a typed query or program over the world model cannot be used at all.

Verification as learning. In this system the decision that something has been learned is the verifier’s, not the teacher’s. That shifts the burden to the evidence. In every recorded case in the lineage where a wrong program was accepted, the evidence itself was wrong, incomplete or vacuous (a lost second demonstration in NL TEACH, a mis-grounded empty evidence set in PDX, truncated or unrepaired demonstrations in HTG, a legal-bad demonstration learned alone before TTP’s Amendment 003).

Completeness versus consistency. The HTG failures show that consistency checks are insufficient: a program can be consistent with every committed trace while the traces omit part of the lesson. TTP’s move was to make completeness a precondition of evidence admission rather than a property to be inferred later. Since then, every committed Arm D demonstration has been behaviorally exact (TTP, both RCI draws and LOCKED); completeness does not protect against a lesson that is complete but semantically wrong, which is the one wrong committed action in RCI (Arm F).

Interactive clarification. Clarification is the channel through which a teacher supplies information that behavior cannot. The worked example shows both sides: the structured question elicited the intended referent even though the trace could not distinguish it, but the controller also clarified when the grounder had already chosen correctly. In RCI the hosted recovery teacher resolved 12 of 12 recovery events in Arm E, while replaying registered rephrasings (Arm B) grounded 0 of 14 recovery slots; in TTP, structured reference choice was exact in 2 of 3 events. These are small numbers, but they suggest that the form of the question matters as much as who answers it.

Teacher availability. The teacher is required to grow the library and not required to use it. That is a real dependency. In the current system, a new skill needs a demonstration from somewhere, and the student cannot generate one for a skill it does not have.

10 Limitations

- **Single author, single executor, no external replication.** Every number was produced under the author’s own preregistered definitions of what counts, executed by one automated system; sealed test sets reduce but do not replace independent replication. The central claim should be read as a strong pilot until at least one sealed set has been run by a different executor. Hosted spend was small (about \$14.6 across the four hosted experiments).
- **The 1.5B model’s role is narrow.** In the current system it is limited to ranking procedure names and writing argument text after handoff, behind deterministic checks.
- **The teacher is in the acquisition chain but not the execution chain;** the paper does not claim that the student learned the teacher’s knowledge.
- **Verification infrastructure is itself subject to the completeness problem** the paper identifies at the semantic layer; the LOCKED teardown race is a case in point (Section 6.1), and the network guard is instrumentation rather than isolation.
- **Synthetic, bounded domain.** One personal-organizer world, a 21-primitive step language, and curricula drawn from 13–15 procedure families plus composites. Requests at reuse time are controlled paraphrases, not open English.
- **Small samples.** Each DEV draw has 36 targets and LOCKED has 23; a perfect LOCKED score is consistent with a system that errs at a few percent per target.
- **LOCKED is not a clean pass.** The preregistered teardown-proof condition failed (Section 6).
- **Strict structural metric.** Learned-exact requires identity with a reference program; behaviorally equivalent alternatives count as failures, which penalizes some correct teaching and depends on the reference programs chosen.
- **Where the language understanding sits.** In the final configuration the hosted 120B model does both teaching and line grounding during acquisition; the frozen 1.5B model’s role is post-handoff request interpretation behind deterministic checks. The results say nothing about the 1.5B model’s ability to understand lessons.
- **Teacher dependence.** Library growth requires a teacher or other source of usable demonstrations.
- **One hosted model family.** Teacher, grounder and judge are one frozen checkpoint in separate roles; results may not transfer to other teachers. vLLM at temperature 0 is not bitwise repeatable, so response caches are the experimental record.
- **Development reuse.** DEV sets were used for engineering within experiments (notably NL_TEACH’s three iterations), so DEV numbers there are optimistic. RCI used two fresh draws with no change between them.
- **Released blind sets.** The sealed LOCKED sets that were never run (NL_TEACH, PDX, HTG/TTP) have their contents withheld, but the HTG/TTP sealed sets were generated from the same template file as the RCI LOCKED set, which is released; they should no longer be treated as blind.

11 Future work

These are hypotheses and plans, not results. The scaling and generality questions determine whether “capability accumulation” is a property of the system or of a small curated domain, so we place them first, alongside the inexpensive LOCKED replication (about \$1 of hosted compute per run) that closes the one formal gap in the current claim. (1) *Retrieval at scale*: name ranking and argument writing with 25, 50,

100 and 250 ACTIVE procedures (libraries can be populated from reference programs without a teacher). (2) *Open-English reuse*: requests beyond the controlled paraphrase distribution, with preregistered diversity buckets and held-out argument values. (3) *Clean LOCKED replication* on a newly sealed set with a strict teardown check, ideally by a different executor. (4) *Cross-domain acquisition*: the same protocol and gates in a second domain, to test whether the slot ledger and lesson form are domain-general. (5) *The boundary of the pattern*: tasks where the correct decomposition is not evident from the goal, so the teacher must invent the structure rather than fill in EXAMPLE/STEP lines; this is where the transactional protocol either extends or breaks. (6) Behavioral rather than structural learned-exact metrics; evidence-diversity requirements (e.g. multi-element witnesses for SORT/FIRST skills); composition depth beyond one; and negative tests for every gate-deciding helper.

12 Conclusion

A frozen small model plus a deterministic runtime acquired new procedures from a temporary 120B teacher and kept them after the teacher was gone. In the final configuration this happened at 70/72 across two independent development draws with no unsafe committed behavior, and at 23/23 on a sealed test set whose preregistered teardown-proof condition nonetheless failed. The lineage suggests that the hard parts were not execution or retention but the teaching channel, and that treating a demonstration as a transaction and verifying programs against evidence carried most of the weight. What the system learns is narrow, inspectable and permanent; how much it can learn still depends on having someone to learn from.

Reproducibility and artifact availability

The repository contains the as-run code of each experiment (not refactored, so that freeze-manifest hashes remain checkable), preregistration documents, locks and manifests, result JSONs, engineering logs, raw teacher, grounder and judge responses keyed by request hash, slot ledgers, teaching transactions, verification records, procedure-store snapshots and handoff and network audits. Reproduction is tiered: (1) fully local unit suites and record replays that need no hosted model (`scripts/run_tests.sh`: 127/127 passing at release; `scripts/verify_rci_rescore.py`: every RCI metric recomputed from the released per-scenario records, 0 differences in 2,530 values); (2) teacher/grounder integration, which requires deploying the same open-weight checkpoint; (3) exact historical results, which are reproduced from the released response caches because temperature-0 serving is not bitwise repeatable. No license file is included; no license has been chosen. See `docs/reproduction.md`.

Use of AI assistance

Experiment code, analysis scripts and drafts of this paper were written with the help of an AI coding assistant working under the author’s direction. All reported numbers are generated by scripts from the released result files.

References

- [1] James F. Allen, Nathanael Chambers, George Ferguson, Lucian Galescu, Hyuckchul Jung, Mary D. Swift, and William Taysom. PLOW: A collaborative task learning agent. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2007.
- [2] Amos Azaria, Jayant Krishnamurthy, and Tom M. Mitchell. Instructable intelligent personal agent. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 2681–2689, 2016.
- [3] Ian Berlot-Attwell, Frank Rudzicz, and Xujie Si. Library learning doesn’t: The curious case of the single-use “library”. In *4th MATH-AI Workshop at NeurIPS*, 2024. arXiv:2410.20274.
- [4] Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. Large language models as tool makers. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2305.17126.

- [5] Allen Cypher, editor. *Watch What I Do: Programming by Demonstration*. MIT Press, 1993.
- [6] Kevin Ellis, Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Lucas Morales, Luke Hewitt, Luc Cary, Armando Solar-Lezama, and Joshua B. Tenenbaum. DreamCoder: Bootstrapping inductive program synthesis with wake-sleep library learning. In *Proceedings of the 42nd ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*, 2021.
- [7] Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. PAL: Program-aided language models. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.
- [8] Gabriel Grand, Lionel Wong, Matthew Bowers, Theo X. Olausson, Muxin Liu, Joshua B. Tenenbaum, and Jacob Andreas. LILO: Learning interpretable libraries by compressing and documenting code. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.19791.
- [9] Alex Graves, Greg Wayne, and Ivo Danihelka. Neural Turing machines. *arXiv preprint arXiv:1410.5401*, 2014.
- [10] Sumit Gulwani. Automating string processing in spreadsheets using input-output examples. In *Proceedings of the 38th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL)*, 2011.
- [11] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*, 2015.
- [12] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. Distilling step-by-step! outperforming larger language models with less training data and smaller model sizes. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.
- [13] Yoon Kim and Alexander M. Rush. Sequence-level knowledge distillation. In *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016.
- [14] James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526, 2017.
- [15] Brenden M. Lake and Marco Baroni. Generalization without systematicity: On the compositional skills of sequence-to-sequence recurrent networks. In *Proceedings of the 35th International Conference on Machine Learning (ICML)*, 2018.
- [16] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.
- [17] Toby Jia-Jun Li, Amos Azaria, and Brad A. Myers. SUGILITE: Creating multimodal smartphone automation by demonstration. In *Proceedings of the 2017 CHI Conference on Human Factors in Computing Systems*, 2017. doi: 10.1145/3025453.3025483.
- [18] Percy Liang, Michael I. Jordan, and Dan Klein. Learning dependency-based compositional semantics. In *Proceedings of the 49th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2011.
- [19] Michael McCloskey and Neal J. Cohen. Catastrophic interference in connectionist networks: The sequential learning problem. In *Psychology of Learning and Motivation*, volume 24, pages 109–165. Academic Press, 1989.
- [20] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [21] Shishir G. Patil, Tianjun Zhang, Xin Wang, and Joseph E. Gonzalez. Gorilla: Large language model connected with massive APIs. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [22] Gordon D. Plotkin. A note on inductive generalization. In *Machine Intelligence 5*, pages 153–163. Edinburgh University Press, 1970.

- [23] Scott Reed and Nando de Freitas. Neural programmer-interpreters. In *International Conference on Learning Representations (ICLR)*, 2016.
- [24] Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [25] Semantic Machines, Jacob Andreas, et al. Task-oriented dialogue as dataflow synthesis. *Transactions of the Association for Computational Linguistics*, 8:556–571, 2020.
- [26] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [27] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [28] Zora Zhiruo Wang, Jiayuan Mao, Daniel Fried, and Graham Neubig. Agent workflow memory. *arXiv preprint arXiv:2409.07429*, 2024.
- [29] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [30] Lifan Yuan, Yangyi Chen, Xingyao Wang, Yi R. Fung, Hao Peng, and Heng Ji. CRAFT: Customizing LLMs by creating and retrieving from specialized toolsets. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17428.
- [31] Luke S. Zettlemoyer and Michael Collins. Learning to map sentences to logical form: Structured classification with probabilistic categorial grammars. In *Proceedings of the 21st Conference on Uncertainty in Artificial Intelligence (UAI)*, 2005.
- [32] Andrew Zhao, Daniel Huang, Quentin Xu, Matthieu Lin, Yong-Jin Liu, and Gao Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2024.

A Lineage details

B Preregistered gates and budgets (RCI)

Arm D pooled gates: learned, scenario and distilled exact $\geq .90$ (minimum counts 65/72, 54/60, 58/64); grounding behavioral exact $\geq .95$ (68/71 given the realized denominator); per-replicate floor .80 on all four. Safety, each replicate: wrong behavioral actions, detectable-bad, legal-bad, conflicting and vacuous activations, verifier false accepts and crashes, incomplete commits, unrequested RETURN and ambiguous references executed all = 0; demonstration completeness and committed-trace exactness = 1. Backend: source replay, counterfactual, rollback = 1; argument binding $\geq .95$; unseen-argument, restart, disconnect and stored reuse given acquisition = 1; hosted teardown verified = 1; post-handoff hosted calls and network attempts = 0; student neural hash invariant. Controls (GOLD_TRACE, FORMAL_TEACH) exact. Controller budgets: 2 clarification and 2 rephrase exchanges per slot, 3 demonstrations and 10 teacher turns per target, 8 teacher turns per demonstration, 2 EXAMPLE requests per demonstration. Grounder: reasoning low, 4,096 tokens, 1 retry.

C Failure taxonomy

Frozen before Stage 0C (spec/FAILURE_TAXONOMY_V1.json). Layers: provider (truncation, transport, runtime); teacher initial (semantic error, missing content, format); grounder initial (op, argument, relation, reference, sequence, scope); recovery (grounder false clarification, grounder/teacher recovery semantic error, teacher recovery missing content or format, clarification/rephrase/teacher-turn/demonstration budget);

Experiment (directory)	Arms / suites	Registered outcome, stop rule
SEMVM_END_TO_END_POC_V1	FULL, GOLD_IR; DEV 30, LOCKED 20	RESULT A; controlled language only (Amendment 001)
SEMVM_PROCEDURE_DISCOVERY_POC_V1	DISCOVERED, GOLD_PROC; DEV v3 30, LOCKED_v2 20	RESULT A; LLM proposal adds nothing over anti-unification
SEMVM_NATURAL_LANGUAGE_PROCEDURE_TEACHING_V1	NL_TEACH, GOLD_TRACE, FORMAL_TEACH; DEV 30 (V1, V1.1, V1.2)	RESULT B + safety stop; LOCKED not run (author ruling)
SEMVM_LLM_TO_SEMVM_PROCEDURAL_DISTILLATION_V1	TEACHER, SCRIPTED, controls; DEV 30	RESULT E/F conditions; LOCKED not run
SEMVM_HOSTED_TEACHER_GROUNDER_V1	A, B, C, D + controls; DEV 30	RESULT G + E; LOCKED not run
SEMVM_TRANSACTIONAL_TEACHING_PROTOCOL_V1	A, B, C, D + controls; DEV2 30 (attempt 2)	RESULT D; LOCKED not run
SEMVM_RECOVERY_CAPACITY_ISOLATION_V1	B, E, F, D + controls; DEV-A 30, DEV-B 30; LOCKED 20 (D + controls)	Stage 1 RESULT A pattern; LOCKED behavioral pass, teardown proof failed

controller (state, reference, evidence count, protocol violation); evidence (insufficient variation, unresolved conflict, non-vacuity failure); procedure (abstraction, retrieval, verifier false accept, false reject, crash). The earliest-cause rule walks a target’s demonstrations chronologically (skipping registered probe demonstrations), takes the first that did not end committed-and-behaviorally-exact, and within it applies the precedence teacher → grounder → controller → evidence → procedure; a provider failure is the cause only if nothing diverged earlier.

D Verifier

Stages 0–6 of `procedure/verify.py`: static validation (types, effects, call depth); interface (the candidate’s parameters match the declared signature); replay of each source trace; counterfactual substitution of unseen values, compared with the unabstracted trace under the same substitution; state perturbations (e.g. all-closed, alternate-closed); negative cases; rollback injection after each state-changing step (the world hash must be unchanged); determinism. The candidate is VERIFIED only if every test passes; activation additionally requires an explicit accept. Registered false-accept probes (frozen value, dropped constant filter, parameterized constant, wrong scope) are run against the same evidence and must be rejected.

E Prompts and protocol cards

The teacher card (sha256 78adbf75...) and grounder card (sha256 6c11956f...) are in `teacher/protocol.py` and `student/hosted_grounder.py`; their hashes are recorded in `locks/FINAL_CONFIG_FREEZE.json`. An excerpt of the teacher card:

```
You are a TUTOR. You teach a small assistant (the STUDENT) a new reusable skill. The
student cannot see your reasoning; it only reads the lines you write, executes them
one by one on its personal-organizer database, and learns the skill from what it
successfully executed.
...
TEACH EXACTLY THE REQUESTED SKILL (goal-faithfulness contract)
- Do not add filters, restrictions, state predicates (open / closed), ordering
  criteria, constants, entity scopes or side conditions unless the skill description
  explicitly requires them, or a saved skill’s signature makes them necessary.
...
```

```
SAVED SKILLS
- To use a saved skill, always write "Call <skill name> with <input> <value>" ...
```

The student’s structured reference question, verbatim from the worked example:

```
For step 2 ("Take the first one.") more than one earlier result fits:
  r1: the requests found (who=Frank, status open) (1 element), from the step "Find the
      open requests with person Frank and sort them by when."
  r2: r1 sorted by when (1 element), from the step "Find the open requests with
      person Frank and sort them by when."
Reply with one line: REFERENT: <id>
```

F Examples

The accepted program of the worked example, as stored in `procedures.sqlite`:

```
PROCEDURE close_oldest_open_request(person:PERSON)
$v0 = FIND REQUEST WHERE SELF.WHO={person} STATUS=OPEN
$v1 = SORT $v0 WHEN
$v2 = FIRST $v1
SET_STATUS $v2 CLOSED
RETURN $v2
END
```

The hosted grounder’s raw output for the ambiguous line (both attempts identical):

```
{"status": "PLAN", "steps": [{"bind": "r3", "op": "FIRST", "of": "r2"}]}
```

Any scenario and arm can be dumped from the released records, and `show_scenario.py` prints a scenario’s curriculum, lessons and reuse turns:

```
python scripts/dump_case.py --suite dev_a --scenario <id> --arm D
python scripts/show_scenario.py dev_a/<id> --arm D
```

G Reproduction

Tier 1 (local, no hosted model): `scripts/run_tests.sh` runs 8 suites (127 tests) and writes `release_tests/SUMMARY.json`; `scripts/verify_rci_rescore.py` recomputes all RCI metrics from records and compares them with the released summaries; `scripts/make_figures_tables.py` regenerates every number, table and data figure in this paper from the released JSONs. Tier 2 (integration): deploy gpt-oss-120b (checkpoint sha256 0fc58fd2...) with vLLM 0.11.0, 32,768 context, and point the evaluator at it; new responses will differ from the cached ones. Tier 3 (historical): the evaluators replay from the role-scoped response caches keyed by request hash, which are the experimental record.

H Invalid runs and amendments

Event	What happened	Disposition
E2E Amendment 001	input-language boundary (controlled language; commands deterministic)	registered before implementation
PDISC DEV v1/v2; LOCKED v1	argument-text failures; one-shot design defect; plan-equivalence tie	general fixes; LOCKED v1 archived unopened; LOCKED_v2 sealed
NL_TEACH V1, V1.1	9 then 2 unsafe executions	Amendments 002, 003; V1.2 final; LOCKED not run
PDX DEV attempts 1, 2	scenario-design defect; 32/98 teacher replies truncated at 1,200 tokens	archived; cap 4,096; final DEV
HTG evaluator defects	judge without credentials; parameterless-reference scoring	evaluator-only rescore; decision unchanged
TTP attempt 1	malformed CHILD crashed an acquisition	archived INVALID; Amendment 002; fresh DEV2
TTP Amendment 003	committed-evidence count	A/C safety invalid; D unaffected; RCI baseline
RCI Stage 0C	teardown targeted wrong app	repaired, 404 verified; acquisition-only
RCI LOCKED	handoff probe 401 accepted as destroyed	strict rescore reported; not rerun

I Cost

Hosted spend (Modal, H100): PDX \$4.78, HTG \$2.58, TTP \$3.62, RCI \$3.57 (Stage 0C \$0.55; DEV-A, DEV-B and LOCKED about \$1 each); E2E \$0.0011 (CPU equivalence check); PDISC and NL_TEACH ran locally (RTX 3060 / 2060). In PDX the hosted teacher used 2.2 turns, 1.2 demonstrations and 1.2 student questions per skill on average, about 0.17 exactly verified skills per 1,000 teacher tokens. In the worked example the lesson call used 1,285 input and 1,129 output tokens and the clarification 1,427 and 216. Procedure libraries are about 50 KB per scenario; the student’s 1.54B parameters are unchanged throughout.

J Project history (background)

This appendix summarizes the preregistered experiments that preceded the seven in Section 4. It is compiled from the author’s consolidated project report of 2026-09-21 and, for the last phase, from the individual result reports. **These experiments’ code and results are not included in this release; the numbers below are quoted as reported and are background, not evidence for the paper’s claims.** A longer version is docs/project_history.md.

Phase (dates, 2026)	Question	Reported finding
1. Synthetic autoregressive reasoning (Aug 13)	can a small model trade inference compute for parameters via executable derivations?	transition competence $\neq$ execution horizon; a token budget is permission to compute, not a policy
2. External associative memory (Aug 27–30)	can knowledge live in a memory beside a small Transformer?	concepts externalize as readable memory; part of the early gain was lexical binding
3. Control/optimization ladder V6–V12 (Aug 31)	why does in-episode derivation fail?	attention supervision saturated without execution gains; the ceiling was undertraining (seed-variable)
4. Macro-core, ml_next, neural instructions (Sep 6–7)	where do real-benchmark traces fail?	GSM8K free 0.376 vs oracle state 0.990: state carry dominates; knowledge scales with memory bytes
5. Semantic front-end on GSM8K (Sep 9–10)	map language to a frozen operator set	operator selection plateaued near 0.56; grounding real text is the frontier
6. Synthetic stateful VM v0–v4C.3 (Sep 11–12)	can a small controller act over explicit durable state?	store scaled 1K→1M records at a constant 32-record working set, final state 1.000 (3 seeds, delexicalized; v2); typed query + typed matching + deterministic projection: relational answers 0.00→1.00 (v4B.1); 64-turn closed loop at ~24 context tokens vs a 1,306-token transcript (v4C)
7. Human language + Wikidata (Sep 12–13)	external knowledge behind a linker	300 facts taught after freezing survived restart at 1.0 with zero optimizer steps; predicate grounding 0.20
8. Frozen code LLMs on SWE-bench (Sep 13–15)	can frozen 0.5B–3B models revise from evidence?	no, at any size; polarity not legible from bounded strings
9. Runtime predicates, real code (Sep 15–16)	can a controller know when it is done?	runtime progress predicates: depth 5–12 episode success 0.063 (0.190 with semantic queries) → 1.0
10. Semantic compilation v6E (Sep 16–17)	compile language to composable records	identity and semantics separable; composition stayed at 0
11. Latent parsers (Sep 17–21)	learn typed records over frozen Qwen features	toy solved (64/64 withheld); real prose fits (811/844) but does not transfer (withheld 0/59)
12a. Typed Qwen frontend (Sep 22–23)	read typed fields from frozen Qwen-0.5B	mid-layer readout 56–58/96 on fresh language; envelope fixed deterministically; no training-side invariance gain
12b. Query-mediated world state (Sep 23–24)	can a frozen backbone + 3.24M head read/write an external store via typed queries?	0.985 exact; 512-entity store at 29.2 tokens/turn; held-out phrasing 0.367 → 0.82 with phrasing diversity (one/two seeds)
12c. MTOP and attachment (Sep 24)	compositional human language	tree exact 0.696 vs 0.512; novel combinations 0.167; novel attachment stuck near 0.80 under LoRA and data fixes
12d. Typed composition (Sep 24–25)	local types, supertags, latent composition	oracle structure assembles exactly; learned predictors fail fit gates; latent factorization collapses
12e. Multi-relation binding (Sep 25–26)	coindexation parent binding	fit failure partly an early-stopping plateau artifact; P0 replay preserves binding (A-005, one seed) → the E2E binder